Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales
Ingeniería en Computación

Redes de Computadoras

Monografía

MQTT y Protocol Buffers

Un stack de comunicación en tiempo real para IoT, robótica y mensajería con servicios en la nube

Equipo
WireGuardians

Viberti, Benjamin[1]	DNI 46224179
Espinoza Sutta, Aaron Alejandro[1]	DNI 96009173
Cleri, Juan Ignacio[1]	DNI 46452662
Pineda, Juan Ignacio[1]	DNI 45591343
Grafión, Atilio Leonel[1]	DNI 43940195
Badenes, Tomás[1]	DNI 44785038
Oviedo, Ignacio Nicolás[1]	DNI 44169239
Mendez, Jorge Nicolas[1]	DNI 41301342

Docente
Oliva Cuneo, Facundo Nicolas

[1]{b.viberti,aaron.espinoza_4500,ignacio.cleri,juan.ignacio.pineda,atilio.grafion,tomasbadenes,
ignacio.oviedo.239,jorge.mendez}@mi.unc.edu.ar

BORRADOR

4 de noviembre de 2026

Índice

Introducción **3**
1. Objetivos 4
2. Alcance 4
   2.1. Tecnologías estudiadas 4
   2.2. Casos de uso y mapeo tecnológico 5
   2.3. Parte experimental 5

Marco teórico **7**
1. Modelo publicador/suscriptor 8
2. MQTT 10
3. Protocol Buffers 12
   3.1. Esquema y generación de código 12
   3.2. Codificación binaria 12
   3.3. Compatibilidad entre versiones 13
   3.4. Limitaciones 13
4. Stack MQTT y Protocol Buffers 13
5. Comparación con alternativas 13
   5.1. MQTT, HTTP y CoAP 14
   5.2. Protocol Buffers y JSON 17

Desarrollo **20**
1. MQTT sobre TCP y TLS 20
   1.1. Conexión persistente y keep-alive 20
   1.2. Cifrado con TLS 20
   1.3. QoS MQTT sobre la confiabilidad de TCP 20
2. Capa de red 20
   2.1. NAT y firewalls 20
   2.2. Ruteo interno 20
   2.3. Ruteo externo 20
   2.4. Convergencia y conexiones de larga duración 20
3. Ruteo por tópicos y ruteo IP 20
4. Casos de uso 20
5. Diseño experimental 20
   5.1. Plataforma común 20
   5.2. Experimento 1: MQTT contra HTTPS para telemetría 21
   5.3. Experimento 2: sincronización de osciladores acoplados 21

Conclusiones **24**
Bibliografía **25**
Anexos **27**
A. Mensajes del experimento de sincronización 27

BORRADOR

Introducción

Gran parte de la Internet comercial se apoya en el modelo de solicitud y respuesta de HTTP: un cliente consulta a un servidor cuando necesita un dato y recibe la respuesta, que suele venir codificada en formatos de texto como HTML, JSON o XML. Este modelo está pensado para intercambios puntuales, que el cliente inicia a demanda, y no para flujos de datos continuos ni para que el servidor tome la iniciativa.

Un número creciente de sistemas no encaja en ese modelo. Los dispositivos del Internet de las Cosas, las flotas de robots y las aplicaciones de mensajería móvil generan flujos continuos de telemetría, eventos y comandos a alta frecuencia y en ambos sentidos, desde equipos de recursos acotados que operan sobre redes celulares o inalámbricas inestables y detrás de firewalls fuera del control de quien administra el servidor. Esta combinación plantea dos limitaciones de fondo para la ingeniería de redes: el costo, en ancho de banda y energía, de consultar periódicamente y de codificar los datos en formatos de texto; y la imposibilidad de que un servidor inicie comunicaciones hacia dispositivos que no admiten conexiones entrantes.

Frente a este panorama, muchos de estos sistemas han recurrido a la mensajería de tipo publicador/suscriptor, que desacopla a los participantes mediante un intermediario. En este campo se inscribe la combinación de MQTT y Protocol Buffers, un stack pensado para la comunicación en tiempo real con ancho de banda limitado. MQTT es un protocolo liviano de transporte de mensajes, organizado en torno a un broker, concebido para entornos restringidos (como los de comunicación máquina a máquina y de Internet de las Cosas) en los que se necesita una huella de código reducida o el ancho de banda es escaso. Protocol Buffers es un mecanismo, independiente del lenguaje y de la plataforma, para serializar datos estructurados y tipados en un formato más compacto y rápido que los textuales, a partir de un esquema compartido.

Su adopción en sistemas reales es amplia y heterogénea. En la mensajería a gran escala, Facebook recurrió a MQTT en Messenger para sostener una conexión persistente con sus servidores sin agotar la batería de los teléfonos, lo que permitió, en muchos casos, entregar los mensajes en cientos de milisegundos en lugar de varios segundos. En robótica, InOrbit, una empresa que ofrece una plataforma de orquestación de flotas de robots, implementa la comunicación entre servicios e integraciones desplegadas *on the edge* utilizando MQTT y mensajes definidos con Protocol Buffers, donde brokers desplegados en la nube reciben datos en tiempo real de miles de robots y dispositivos IoT, que a su vez son controlados por el mismo canal, aun en instalaciones con redes inestables. Desarrollado por la misma empresa, el producto de código abierto OpenRobOps libera las definiciones de su protocolo de comunicación en estructuras Protobuf que fijan el formato de las comunicaciones entre robot y plataforma.

Un rasgo de este esquema es especialmente relevante desde el encaminamiento: el dispositivo abre una conexión persistente hacia el broker alojado en la nube, cuya dirección suele estar habilitada en el firewall, y por esa misma conexión el broker le entrega los mensajes, sin necesidad de conexiones entrantes. El principio rector del stack es, entonces, trasladar la complejidad desde la red hacia el protocolo de aplicación: ningún participante necesita conocer la ubicación de los demás, sino solo la del broker, y el contrato sobre el formato de los datos se fija una única vez en un esquema distribuido entre los clientes. De este modo, la comunicación en tiempo real entre miles de entidades heterogéneas resulta viable sobre redes que no fueron concebidas para ese fin.

BORRADOR

1. Objetivos

Se desea realizar un análisis riguroso del stack de comunicación formado por MQTT y Protocol Buffers, como alternativa para el intercambio de datos en tiempo real, a alta frecuencia y con ancho de banda limitado, en aplicaciones de Internet de las Cosas, robótica y mensajería. El estudio sigue el recorrido de un mensaje desde el dispositivo que lo genera hasta el broker que lo distribuye, y evalúa cómo cada capa de la arquitectura contribuye a que ese recorrido sea eficiente y confiable.

Más que catalogar las prestaciones de ambas tecnologías, el trabajo busca explicar por qué resultan adecuadas para el problema planteado: sostener una comunicación continua sobre redes que el diseñador no controla, con enlaces inestables, firewalls de terceros y trayectos que atraviesan múltiples sistemas autónomos. Con ese fin, el análisis se articula en torno a dos ejes centrales de la materia, que pertenecen a capas distintas: la capa de aplicación (capa 7), donde operan MQTT y Protocol Buffers, y la capa de red (capa 3), donde el ruteo determina por qué camino llegan los mensajes al broker y cómo se restablece ese camino cuando la red cambia.

Para alcanzar este propósito se plantean los siguientes objetivos específicos:

- Presentar el modelo publicador/suscriptor y describir cómo lo implementa MQTT (broker, tópicos, niveles de calidad de servicio y sesiones), junto con el mecanismo de serialización de Protocol Buffers, y compararlos con alternativas como HTTP, CoAP y JSON.
- Analizar el funcionamiento de MQTT sobre la capa de transporte (TCP, keep-alive y TLS) y distinguir las garantías de entrega de sus niveles de calidad de servicio de la confiabilidad que ya ofrece TCP.
- Analizar el recorrido de los mensajes en la capa de red (capa 3): NAT y firewalls, y el efecto de la convergencia de los protocolos de ruteo interno (OSPF, con RIP y EIGRP como comparación) y externo (BGP) sobre las conexiones TCP de larga duración en las que se apoya MQTT.
- Contrastar el ruteo por destino IP de la capa de red con el ruteo por tópicos que realiza el broker en la capa de aplicación.
- Examinar casos de uso reales en robótica, mensajería e Internet de las Cosas, como los de InOrbit, para determinar las ventajas y limitaciones del stack en entornos de alta frecuencia y recursos acotados.
- Medir experimentalmente, sobre nodos emulados, el costo en bytes y latencia de MQTT y Protocol Buffers frente a HTTPS y JSON, y su comportamiento ante pérdidas, latencia y cortes de conexión.

2. Alcance

El trabajo se centra en el stack formado por MQTT y Protocol Buffers, considerando distintos transportes para MQTT (TCP, TLS y WebSockets), y en el recorrido de un mensaje desde el dispositivo que lo genera hasta el broker que lo distribuye. A continuación se precisa qué aspectos abarca el análisis y cuáles quedan fuera de él.

2.1. Tecnologías estudiadas

En la capa de aplicación, el estudio considera MQTT en su versión 5.0 y Protocol Buffers en su sintaxis proto3. Del protocolo MQTT se analizan el modelo de broker, los tópicos y sus comodines, los niveles de calidad de servicio, las sesiones, los mensajes retenidos y el *last will*. De Protocol Buffers se analizan el esquema, la codificación binaria y la compatibilidad entre

BORRADOR

versiones. Como puntos de comparación se toman HTTP (y HTTPS) y CoAP como protocolos de transporte de mensajes, y JSON como formato de serialización.

En la capa de transporte, el análisis se limita a TCP y TLS en la medida en que afectan a MQTT, conceptos como la conexión persistente, el keep-alive, el costo del establecimiento de la sesión segura y la relación entre la confiabilidad de TCP y los niveles de calidad de servicio de MQTT. La seguridad se aborda únicamente desde el cifrado del canal y la autenticación de los clientes, sin un análisis integral de amenazas.

En la capa de red, se estudian NAT y los firewalls, y el ruteo interno y externo por el que los mensajes llegan al broker. Para el ruteo interno se toma OSPF como protocolo principal y se usan RIP y EIGRP como comparación. Para el ruteo externo se considera BGP. Estos protocolos se analizan en cuanto a su convergencia y su efecto sobre las conexiones TCP de larga duración, y no en cuanto a su configuración en equipos concretos.

2.2. Casos de uso y mapeo tecnológico

Los casos de uso se toman de fuentes públicas, entre ellas la mensajería de Facebook Messenger y las plataformas de robótica de InOrbit y OpenRobOps, y se analizan a partir de la información que publican las propias empresas, sin acceso a sus implementaciones internas. Además, el trabajo incluye un mapeo tecnológico con los equipos reales con los que se implementaría el sistema estudiado: dispositivos de borde, el broker (en un servidor propio o como servicio administrado) y la conectividad. Para cada uno se indica el modelo, el proveedor y un precio de referencia con su fecha y moneda. El objetivo es estimar la viabilidad de una solución considerando la calidad técnica, el plazo y el presupuesto, y no proponer la alternativa de máxima prestación.

2.3. Parte experimental

Los experimentos se realizan sobre placas ESP32 emuladas con QEMU, que se comunican con un broker Mosquitto y un backend, estos cumplen la función de contrastar con mediciones propias lo que el marco teórico y el desarrollo plantean de forma cualitativa. El primero cuantifica la ventaja en bytes y en latencia que la bibliografía atribuye a MQTT y a Protocol Buffers, y permite separar cuánto aporta cada uno y cuánto cuesta el cifrado. El segundo ilustra el comportamiento del stack en una aplicación de tiempo real con intercambio a alta frecuencia, en la que las pérdidas, la latencia y los cortes de conexión analizados en el desarrollo tienen un efecto directo y observable sobre el resultado.

Se plantean dos experimentos en los que se comparan las siguientes combinaciones de protocolo y formato de serialización:

- MQTT con Protocol Buffers
- MQTT con JSON
- HTTP con Protocol Buffers
- HTTP con JSON

El primer experimento se realiza en un escenario de telemetría y mide por separado el costo en bytes y en latencia del transporte y de la serialización. El segundo compara las mismas combinaciones en la sincronización de osciladores acoplados. En ambos, las condiciones de red, como las pérdidas, la latencia y los cortes de conexión, se introducen de forma artificial, y cada combinación se evalúa con y sin TLS.

La emulación impone limitaciones que acotan el alcance de los resultados. No reproduce el comportamiento de una radio real, por lo que no se mide el consumo de energía de los dispositivos

BORRADOR

y no se incluyen enlaces inalámbricos restringidos como LoRa, NB-IoT o satelitales. Tampoco reproduce los tiempos de procesamiento de un microcontrolador físico. Los escenarios de ruteo se emulan, en lugar de reproducirse sobre una red con múltiples sistemas autónomos reales. Por último, no se evalúan la escalabilidad del broker con miles de clientes ni las configuraciones en clúster.

BORRA-
DOR

Marco teórico

Las redes de computadoras existen para hacer posibles las aplicaciones de red, es decir, programas que se ejecutan en distintos sistemas terminales y que se comunican entre sí a través de la red. La lógica de estas aplicaciones reside únicamente en los sistemas terminales, mientras que los nodos intermedios operan en la capa de red e inferiores. En consecuencia, quien desarrolla una aplicación no modifica la red, sino que utiliza los servicios que esta ofrece y define cómo se organiza la comunicación entre los extremos. MQTT y Protocol Buffers se ubican en este plano.

La forma predominante de organizar la comunicación a nivel de aplicación es el modelo cliente-servidor [1]. En este modelo, un servidor permanece activo en una dirección conocida a la espera de solicitudes, y un cliente lo contacta cuando requiere un servicio, envía una solicitud y aguarda la respuesta. La Web constituye el ejemplo característico, en el que el navegador solicita un recurso y el servidor lo devuelve. Se trata de un modelo orientado a la demanda, en el que la información se transmite únicamente cuando es solicitada y el intercambio es iniciado siempre por la parte que la requiere.

Este modelo presenta limitaciones para los sistemas considerados en este trabajo. En ellos, los dispositivos generan datos de manera continua y en instantes que no pueden anticiparse, y deben a su vez recibir comandos originados en la nube. Para que el servidor transmita un comando por iniciativa propia, debería actuar como cliente del dispositivo, lo que requiere que este acepte conexiones entrantes. Esta condición rara vez se cumple cuando el dispositivo se encuentra detrás de un NAT o de un firewall administrado por terceros. La alternativa que ofrece el propio modelo es la consulta periódica por parte del dispositivo, cuyo costo en tráfico y en latencia aumenta con la frecuencia de las consultas.

La capa de transporte ofrece una base para superar esta limitación. TCP establece una conexión persistente *full-duplex*, que permite a ambos extremos transmitir datos en cualquier momento con entrega fiable y ordenada. Si es el dispositivo quien abre la conexión hacia un servidor de dirección conocida, no necesita aceptar conexiones entrantes y conserva el rol de cliente. En ese caso, una vez establecida la conexión, el servidor también puede enviarle mensajes al cliente sin esperar que este realice una solicitud, lo que vuelve innecesaria la consulta periódica.

La conexión TCP, sin embargo, solo garantiza la entrega de una secuencia de bytes entre dos procesos. Qué mensajes se intercambian, con qué formato y en qué momento lo determina el protocolo de la capa de aplicación, que especifica los tipos de mensajes, su sintaxis y su semántica, así como las reglas que indican cuándo y cómo se envía y se responde cada uno de ellos [2].

En los sistemas estudiados, el diseño de ese protocolo está condicionado en primer lugar por la forma en que circula la información. Un mismo dato suele interesar a varios consumidores y cada consumidor recibe datos de muchos dispositivos, por lo que resulta conveniente que productores y consumidores puedan comunicarse sin conocerse entre sí ni estar conectados simultáneamente, que es la propuesta del modelo publicador/suscriptor implementado por MQTT. A esto se suma la representación de los datos dentro de cada mensaje, que debe ser interpretada de manera unívoca por equipos heterogéneos y ocupar poco ancho de banda en enlaces que suelen ser limitados, problema del que se ocupa Protocol Buffers.

BORRADOR

1. Modelo publicador/suscriptor

El modelo publicador/suscriptor (*publish/subscribe*) ofrece precisamente la posibilidad de que productores y consumidores de información se comuniquen sin conocerse entre sí, lo que lo hace adecuado para los sistemas descritos. En este esquema de interacción, los suscriptores expresan su interés en un evento o en un patrón de eventos, y son notificados de forma asíncrona de todo evento generado por un publicador que coincida con ese interés [3]. El modelo básico se apoya en un servicio de notificación de eventos, una abstracción que almacena las suscripciones y entrega cada evento publicado a todos los suscriptores interesados. Las suscripciones permanecen en ese servicio y son invisibles a los publicadores.

De esta manera, una de las características principales del modelo y común a todas sus variaciones es el desacoplamiento total de las entidades que se comunican, que puede descomponerse en tres dimensiones [3]:

- El **desacoplamiento en el espacio** (*space decoupling*) implica que las partes no necesitan conocerse, ya que los publicadores no mantienen referencias a los suscriptores ni saben cuántos participan de la interacción, y lo mismo ocurre en sentido inverso.
- El **desacoplamiento en el tiempo** implica que las partes no necesitan participar de la interacción al mismo tiempo, por lo que un publicador puede publicar eventos mientras un suscriptor está desconectado, y un suscriptor puede recibir la notificación de un evento cuyo publicador ya se desconectó.
- El **desacoplamiento de sincronización** implica que los publicadores no se bloquean al producir eventos y que los suscriptores son notificados mientras realizan otras actividades, fuera del flujo principal de control de ambos.

Este desacoplamiento reduce la coordinación necesaria entre los dispositivos participantes de la comunicación y favorece la escalabilidad en entornos asíncronos por naturaleza.

Estas tres dimensiones permiten contrastar el modelo con el cliente/servidor descrito al comienzo del capítulo. En el caso del paradigma cliente/servidor el cliente debe conocer la dirección del servidor, ambos deben estar activos durante el intercambio y el cliente queda a la espera de la respuesta, por lo que la interacción está acoplada en el espacio, en el tiempo y, del lado del cliente, en la sincronización. La misma comparación puede extenderse a otros paradigmas de comunicación distribuida, como resume la Tabla 1. La invocación remota de procedimientos (RPC) presenta el mismo acoplamiento que el modelo cliente-servidor, mientras que sus variantes asíncronas y el patrón *observer* eliminan únicamente el acoplamiento de sincronización. Los espacios de tuplas y las colas de mensajes desacoplan a productores y consumidores en el espacio y en el tiempo, pero obligan a los consumidores a extraer los datos de forma síncrona. El modelo publicador/suscriptor es el único que reúne las tres propiedades.

BORRA-DOR

Tabla 1: Desacoplamiento que ofrece cada paradigma de comunicación [3, p. 121].

Paradigma	**Espacio**	**Tiempo**	**Sincronización**
Pasaje de mensajes	No	No	Solo el productor
RPC/RMI	No	No	Solo el productor
RPC/RMI asíncrona	No	No	Sí
RPC/RMI con futuros	No	No	Sí
Notificaciones (*observer*)	No	No	Sí
Espacios de tuplas	Sí	Sí	Solo el productor
Colas de mensajes (*pull*)	Sí	Sí	Solo el productor
Publicador/suscriptor	Sí	Sí	Sí

Las variantes del modelo publicador/suscriptor se distinguen por la forma en que los suscriptores especifican los eventos de interés. En el esquema basado en tópicos, el más antiguo, los eventos se publican bajo nombres que los sistemas modernos organizan en jerarquías, y la sintaxis de comodines permite suscribirse a todos los tópicos de la jerarquía que coincidan con el patrón dado. Este esquema puede implementarse de manera muy eficiente, aunque puede resultar limitado. El esquema basado en contenido clasifica los eventos según sus propias propiedades, mediante filtros que define el suscriptor, y ofrece mayor expresividad a costa de protocolos más complejos y de un mayor costo de ejecución. El esquema basado en tipos clasifica los eventos según su tipo, lo que permite verificar en tiempo de compilación que productores y consumidores coinciden en la estructura de los datos. MQTT, descrito en la Sección 2, adopta el esquema basado en tópicos.

La implementación del servicio de eventos admite a su vez distintas arquitecturas. En una arquitectura centralizada, los productores envían los mensajes a una entidad que los almacena y los reenvía a los consumidores. En una arquitectura distribuida no existe tal entidad, y los mecanismos de almacenamiento y reenvío residen en los propios procesos de productores y consumidores, mientras que un enfoque intermedio implementa el servicio como una red de servidores dedicados. En la arquitectura centralizada, que es la que adopta MQTT, la entidad central se denomina *broker*. Este suele comunicarse con publicadores y suscriptores mediante canales punto a punto confiables y conservar copias de los eventos en almacenamiento estable, lo que le permite ofrecer persistencia y entrega garantizada. Como contraparte, el uso de un broker agrega un salto a cada mensaje y concentra la carga de todos los participantes en un punto de falla que tiene el potencial de demorar la entrega de los eventos [3].

Sobre una red TCP/IP, la arquitectura centralizada se apoya en interacciones cliente-servidor. Tanto los publicadores como los suscriptores establecen la conexión hacia el broker, por lo que todos ellos actúan como clientes, y el broker es el único participante que debe tener una dirección conocida y aceptar conexiones entrantes. Su función es la de un sistema intermedio que mantiene conexiones TCP con cada extremo y retransmite los mensajes entre ellas, de manera análoga a los intermediarios definidos para HTTP [4]. La diferencia con el modelo cliente-servidor aparece una vez registrada la suscripción, ya que el broker entrega cada evento por la conexión que el suscriptor estableció previamente, sin requerir de una solicitud. De este modo, los datos llegan al suscriptor en el momento en que se producen, sin que este deba consultar periódicamente ni aceptar conexiones entrantes. Esta capacidad de entregar datos por iniciativa del otro extremo es común a cualquier canal bidireccional persistente, como una conexión WebSocket, pero lo que distingue al modelo publicador/suscriptor es que, sobre esos canales, el broker determina a

BORRADOR

qué suscriptores corresponde cada evento de modo que publicadores y suscriptores permanecen desacoplados aun cuando cada uno mantiene una conexión punto a punto con el broker.

2. MQTT

MQTT es un protocolo de transporte de mensajes de tipo publicador/suscriptor, basado en una arquitectura cliente-servidor y diseñado para ser liviano, abierto y fácil de implementar [5]. La primera versión estandarizada por OASIS fue la 3.1.1, adoptada también como norma ISO/IEC 20922, y la versión 5.0, en la que se centra este trabajo, incorpora numerosas funcionalidades nuevas sin alterar el núcleo del protocolo. MQTT adopta la arquitectura centralizada descrita en la sección anterior denominando *broker* al servidor y cliente a todo programa o dispositivo que abre una conexión hacia él para publicar mensajes y/o suscribirse a ellos, y deja a cada implementación la elección de los mecanismos de autenticación, autorización y cifrado. MQTT delega en la capa de transporte la entrega ordenada y sin pérdidas de los datos a la capa de transporte, por lo que funciona sobre TCP, TLS o WebSocket, pero no directamente sobre UDP.

Cada mensaje de aplicación lleva un nombre de tópico, el cual consiste en una cadena de texto que el servidor compara con las suscripciones de los clientes. Los tópicos se organizan en niveles separados por el carácter `/`, y las suscripciones se expresan mediante filtros que se comparan con los tópicos de mensajes entrantes. Las expresiones de suscripción pueden incluir los comodines `+`, que coincide con exactamente un nivel, y `#`, que solo puede ubicarse al final del filtro y coincide con cualquier cantidad de niveles.

De esta manera las expresiones de suscripción permiten, por ejemplo, en una flota de robots que publica en tópicos de la forma `flota/{ID de robot}/{sensor}`, suscribirse con el filtro `flota/+/bateria` que recibe el estado de batería de todos los robots o el filtro `flota/robot1/#`, que recibe todos los mensajes de el robot `robot1`.

Implementar el esquema basado en tópicos, MQTT implementa desacoplamiento en el espacio, ya que el publicador solo conoce el nombre del tópico y es el servidor quien determina qué clientes reciben cada mensaje. La versión 5.0 agrega además las suscripciones compartidas, en las que cada mensaje se entrega a uno solo de los clientes de un grupo, lo que permite repartir el procesamiento entre varios consumidores.

La entrega de los mensajes se rige por tres niveles de calidad de servicio (QoS), cuyos intercambios de paquetes muestra la Figura 1. Con QoS 0, el mensaje se envía sin confirmación ni reintentos y llega una vez o ninguna. Con QoS 1, el receptor confirma cada mensaje con un paquete PUBACK y el emisor lo conserva hasta recibir esa confirmación, lo que asegura que llegue al menos una vez aunque pueden producirse duplicados. Con QoS 2, un intercambio en dos etapas garantiza que el mensaje llegue exactamente una vez, a costa de una mayor sobrecarga.

El nivel de QoS se aplica por separado entre el publicador y el broker y entre el broker y cada suscriptor, y cada suscriptor recibe el mensaje con el menor nivel de QoS entre el usado al publicarlo y el máximo concedido a su suscripción. La relación entre estos niveles y la confiabilidad que ya ofrece TCP se analiza en la Sección 1.3.

BORRADOR

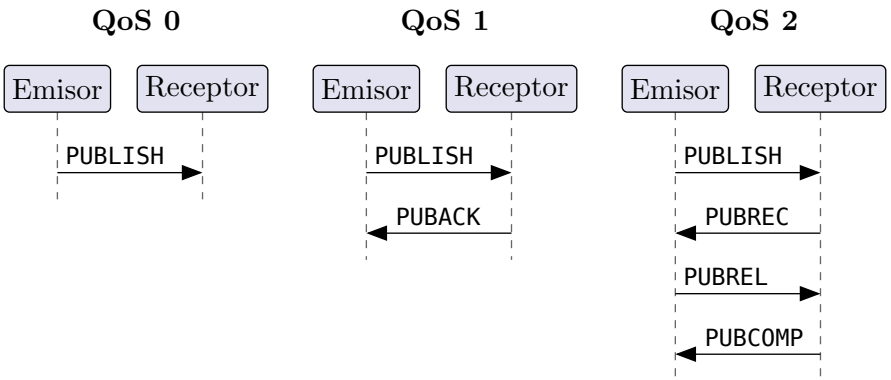


Figura 1: Intercambio de paquetes de control en cada nivel de QoS.

Las confirmaciones de QoS 1 y 2 requieren que cliente y servidor mantengan un estado de sesión. En el servidor, ese estado comprende las suscripciones del cliente y los mensajes destinados a él que aún no fueron enviados o confirmados, y puede conservarse a lo largo de varias conexiones consecutivas. Al conectarse, el cliente indica si inicia una sesión nueva o retoma la anterior, y durante cuánto tiempo debe conservarse el estado una vez cerrada la conexión. Este mecanismo es el que permite el desacoplamiento en el tiempo, ya que el servidor almacena los mensajes dirigidos a un suscriptor desconectado y se los entrega cuando este vuelve a conectarse. Sin embargo, ese desacoplamiento no es incondicional, ya que si la sesión no persiste más allá de la conexión, o si los mensajes se publican con QoS 0 que el servidor no está obligado a almacenar, un suscriptor desconectado nunca recibe los mensajes publicados, por lo que corresponde a la aplicación elegir la combinación adecuada para cada tipo de dato.

Complementando este comportamiento, MQTT suma dos mecanismos más. Un mensaje publicado con el indicador RETAIN se almacena en el servidor como último mensaje retenido de su tópico y se entrega a cada nuevo suscriptor, de modo que un cliente que se suscribe recibe de inmediato el estado más reciente aunque el publicador lo envíe de forma irregular. El mensaje de última voluntad (*Will Message*), por su parte, lo registra el cliente al conectarse y el servidor lo publica a los suscriptores si la conexión se cierra de forma anormal, por ejemplo cuando el cliente deja de comunicarse dentro del intervalo de keep-alive, que es el tiempo máximo que puede transcurrir entre dos paquetes enviados por el cliente. Esto permite notificar a los interesados de la desconexión de un dispositivo.

Por último, toda la comunicación en MQTT se realiza mediante paquetes de control, de los cuales la especificación define quince tipos. Entre ellos se encuentran CONNECT y CONNACK para establecer la conexión, PUBLISH para transportar los mensajes de aplicación, PUBACK y los demás paquetes de confirmación de los niveles de QoS, SUBSCRIBE y UNSUBSCRIBE para gestionar las suscripciones, y PINGREQ y PINGRESP para el keep-alive. Cada paquete se compone de un encabezado fijo, presente en todos ellos, seguido de un encabezado variable y de una carga útil que solo aparecen en algunos tipos. El primer byte del encabezado fijo indica el tipo de paquete y algunos indicadores, como el nivel de QoS y RETAIN en el caso de PUBLISH, y a continuación se codifica la longitud restante del paquete con un entero de longitud variable que ocupa un solo byte para valores de hasta 127. De este modo, el encabezado fijo ocupa apenas dos bytes en los paquetes pequeños, que es también el tamaño total de los paquetes de keep-alive. El uso de este tipo de encabezados resulta en una economización del costo de transporte, la cual se evalúa en el experimento de la Sección 5.2.

El contenido de los mensajes queda fuera del protocolo MQTT, que transporta la carga útil sin interpretarla. La versión 5.0 permite declarar su formato mediante las propiedades de indicador

BORRADOR

de formato y tipo de contenido, que el servidor reenvía sin modificar, pero no impone ninguno, aspecto que se retoma en la Sección 4. La representación de los datos queda así a cargo de la aplicación, que es el problema que aborda Protocol Buffers, descrito en la Sección 3.

3. Protocol Buffers

Una conexión TCP entrega una secuencia de bytes, y un broker MQTT reenvía el contenido de cada mensaje sin interpretarlo. Para que un dato producido por un microcontrolador sea leído correctamente por un servicio escrito en otro lenguaje y ejecutado en otra arquitectura, ambos extremos deben acordar cómo se traduce una estructura de datos a bytes y viceversa, incluyendo el orden de los bytes, el tamaño de los enteros y la forma de delimitar cada campo. En el modelo OSI esta tarea corresponde a la capa de presentación [4], pero la arquitectura TCP/IP no la incluye, por lo que cada aplicación debe resolverla por su cuenta [1].

Protocol Buffers, también conocido como Protobuf, es un mecanismo para serializar datos estructurados que es neutral respecto del lenguaje de programación y de la plataforma [6]. Fue desarrollado por Google, que lo publicó como software libre y mantiene implementaciones oficiales para una variedad de lenguajes [7]. Su propuesta es comparable a la de JSON, pero con mensajes más pequeños, un procesamiento más rápido y la generación de código nativo para cada lenguaje. Según su documentación, ofrece un formato de serialización para paquetes de datos estructurados y tipados de hasta unos pocos megabytes, adecuado tanto para el tráfico de red efímero como para el almacenamiento de datos a largo plazo, y que puede extenderse con nueva información sin invalidar los datos existentes ni obligar a actualizar el código [6]. Estas propiedades explican su amplia adopción por parte de grandes empresas de internet y en arquitecturas de microservicios [8].

3.1. Esquema y generación de código

La estructura de los datos se define una única vez en un archivo de esquema con extensión `.proto`, en el que cada mensaje se describe como un conjunto de campos con un tipo, un nombre y un número que lo identifica. Por ejemplo, la lectura de un sensor ambiental podría definirse así:

```
syntax = "proto3";

message Lectura {
  float temperatura = 1;
  float humedad     = 2;
  uint32 bateria    = 3;
}
```

A partir de este archivo, el compilador `protoc` genera el código fuente que permite construir, serializar y deserializar el mensaje en cada lenguaje de destino. De este modo, el `.proto` cumple a la vez la función de documentación y la de contrato explícito entre los equipos o servicios que intercambian los datos, y un mismo esquema produce implementaciones compatibles en todos los lenguajes soportados.

3.2. Codificación binaria

En la codificación binaria de Protobuf, cada campo se transmite precedido de una etiqueta (*tag*) que combina el número de campo definido en el esquema con un tipo de codificación (*wire type*), que indica cómo leer el valor a continuación, como un entero de longitud variable, un valor de

BORRADOR

32 o de 64 bits de longitud fija, o un bloque precedido de su longitud, que se usa para cadenas, bytes y mensajes anidados [9]. Se trata, en esencia, de una codificación de tipo-longitud-valor (TLV), como la de BER.

Los enteros se codifican como *varints*, en los que cada byte aporta siete bits del valor y utiliza el bit más significativo para indicar si el número continúa en el byte siguiente. Así, los valores pequeños, que son los más frecuentes, ocupan un solo byte. Por ejemplo, un campo número 1 con el valor 150 se codifica en tres bytes, `08 96 01`: el primero es la etiqueta y los dos restantes el valor [9]. Para los enteros con signo existe además la codificación *zigzag*, que intercala positivos y negativos para que los valores negativos pequeños también resulten cortos. El mensaje no incluye los nombres de los campos, que solo existen en el esquema, y en proto3 los campos que tienen su valor por defecto directamente no se transmiten [10]. Esta combinación es la que hace que un mensaje Protobuf pese varias veces menos que su equivalente en JSON y que se procese más rápido.

3.3. Compatibilidad entre versiones

Dado que los campos se identifican por número y no por posición ni por nombre, el esquema puede evolucionar sin romper a quienes usan versiones anteriores. Un programa que recibe un campo cuyo número no conoce simplemente lo ignora, de modo que los clientes viejos pueden leer mensajes nuevos (compatibilidad hacia adelante). En sentido inverso, un programa nuevo que lee un mensaje viejo encuentra los valores por defecto en los campos ausentes (compatibilidad hacia atrás). La condición para que esto funcione es no reutilizar números de campo, para lo cual el lenguaje ofrece la palabra clave `reserved`, que impide volver a asignar los números o nombres de campos eliminados [10]. Esta compatibilidad en ambos sentidos es la principal fortaleza de Protobuf frente a otros formatos binarios.

3.4. Limitaciones

Las mismas decisiones que reducen el tamaño de los mensajes tienen costos. En primer lugar, el formato no es legible por personas, por lo que la depuración requiere herramientas como `protoc --decode` o conversores a JSON. En segundo lugar, no es autodescriptivo: sin el esquema no es posible interpretar los datos, ya que el binario no indica qué mensaje contiene ni qué significa cada campo. En tercer lugar, la codificación no delimita los mensajes, por lo que el protocolo que los transporta debe indicar dónde termina cada uno. Por último, agrega al proyecto un paso de compilación y de generación de código, y resulta menos práctico que JSON para APIs públicas simples o para su uso directo desde navegadores, aunque para ese caso existen alternativas como gRPC-Web.

4. Stack MQTT y Protocol Buffers

5. Comparación con alternativas

Para intercambiar datos entre un dispositivo (ya sea un sensor, un celular o un robot) y la nube deben tomarse dos decisiones:

1. Qué protocolo de capa de aplicación implementar (define cómo se intercambian los mensajes).
2. Qué mecanismo de serialización se utiliza para representar los datos dentro de cada mensaje (con qué formato viaja la información).

Las secciones anteriores describieron la combinación de MQTT y Protocol Buffers como respuesta a ambas decisiones para el intercambio de datos en tiempo real con ancho de banda

BORRADOR

limitado, pero no es la única opción, ya que existen otras alternativas consolidadas. Esta sección analiza esas alternativas para identificar en qué aspectos MQTT y Protocol Buffers aventajan a cada una de ellas y en cuáles resultan inferiores. En particular, se compara MQTT con otros dos protocolos de capa de aplicación: HTTP, sobre el que se apoya gran parte de la Internet comercial, y CoAP, diseñado específicamente para dispositivos restringidos. Del mismo modo, se compara Protocol Buffers con JSON, un mecanismo de serialización basado en texto y de uso extendido.

La comparación de protocolos considera el modelo de interacción, el protocolo de transporte, el tamaño de los encabezados, la confiabilidad y el paso por NAT y firewalls, mientras que la de mecanismos de serialización considera el tamaño de los mensajes, el costo de procesamiento y la necesidad de un esquema compartido. El análisis se apoya en las especificaciones de cada tecnología y en estudios publicados.

5.1. MQTT, HTTP y CoAP

Se definirán brevemente los protocolos HTTP y CoAP, para posteriormente adentrarse en sus diferencias con MQTT.

5.1.1. HTTP

Se trata de un protocolo de aplicación **sin estado** de **solicitud y respuesta**. Cada solicitud se interpreta de manera independiente, por lo que el servidor no conserva información sobre los clientes entre una solicitud y la siguiente [2], [4, p. 780]. Es el protocolo de la Web y el ejemplo característico del modelo cliente-servidor: el cliente envía una solicitud que indica un método (como GET o POST) y el recurso al que se refiere (identificado por una URI), y el servidor devuelve una respuesta con un código de estado y, eventualmente, el contenido solicitado [11].

HTTP utiliza TCP como protocolo de transporte, del que obtiene la entrega fiable y ordenada de los datos [2]. En el modelo de solicitud y respuesta, el servidor no puede iniciar una conexión con el cliente ni enviarle una respuesta que no haya sido solicitada. Por eso, un cliente que necesita enterarse de eventos nuevos debe consultar periódicamente al servidor, técnica conocida como *polling* [12].

5.1.2. CoAP

CoAP (*Constrained Application Protocol*) es un protocolo de transferencia web estandarizado por el IETF (*Internet Engineering Task Force*) para nodos y redes restringidos, como microcontroladores con poca memoria que operan sobre redes de bajo consumo y con pérdidas. Está orientado a aplicaciones máquina a máquina (M2M) [13].

Adopta el mismo modelo de solicitud y respuesta que HTTP, con recursos identificados por URI, métodos como GET, POST, PUT y DELETE, y códigos de respuesta. Sin embargo, no busca comprimir HTTP, sino implementar el subconjunto de esa arquitectura que necesitan las aplicaciones M2M, y está diseñado para traducirse fácilmente a HTTP mediante *proxies*, lo que facilita su integración con la Web [13].

CoAP funciona sobre UDP, con DTLS como mecanismo de seguridad, y codifica sus mensajes en un formato binario compacto [13]. Como UDP no garantiza la entrega, el propio protocolo incorpora una confiabilidad liviana: los mensajes confirmables (CON) se retransmiten hasta que el receptor responde con un acuse de recibo (ACK), mientras que los no confirmables (NON) se envían sin confirmación, lo que resulta adecuado, por ejemplo, para cada lectura de un flujo de datos de un sensor [13].

BORRADOR

Por último, la extensión Observe permite que un cliente se registre en un recurso para que el servidor le envíe una nueva representación cada vez que ese recurso cambia, lo que evita el *polling* [14].

5.1.3. Comparación

De las descripciones anteriores y de los estudios que comparan estos protocolos se desprenden las siguientes diferencias:

1. **Modelo de interacción.** HTTP y CoAP siguen el modelo de solicitud y respuesta, en el que el cliente aplica métodos sobre recursos y recibe códigos de respuesta. En ese modelo, el dato viaja cuando el cliente lo pide, por lo que para enterarse de eventos nuevos debe recurrir al *polling*, cuyo costo crece con la frecuencia de las consultas, o al *long polling*, en el que cada notificación sigue siendo un mensaje HTTP completo, con todos sus encabezados [12]. La extensión Observe acerca CoAP a un modelo de notificaciones, pero sin un broker que desacople a las partes, y su propia especificación aclara que no pretende reemplazar a los sistemas publicador/suscriptor [14]. MQTT, en cambio, implementa el modelo publicador/suscriptor: el broker entrega cada mensaje en el momento en que se publica, mediante la conexión que el suscriptor estableció previamente, y puede conservar los mensajes destinados a un cliente desconectado mediante sesiones persistentes y mensajes retenidos [5], algo que HTTP y CoAP no prevén.
2. **Protocolo de transporte.** HTTP y MQTT funcionan sobre TCP, del que heredan la entrega fiable y ordenada, pero deben establecer una conexión antes de intercambiar datos. MQTT la establece una sola vez y la mantiene abierta, mientras que HTTP puede abrir una conexión por cada solicitud o reutilizarla para varias [2]. CoAP, en cambio, funciona sobre UDP y no necesita establecer una conexión, de modo que una transacción se resuelve con dos datagramas, uno en cada sentido [15].
3. **Tamaño de los encabezados.** HTTP utiliza encabezados de texto de longitud variable, mientras que los de CoAP y MQTT son binarios: el encabezado fijo de CoAP ocupa cuatro bytes [13] y el de MQTT, dos bytes como mínimo [5]. Sin embargo, la conexión TCP que requiere MQTT aumenta su sobrecarga total. Por eso, en las comparaciones cualitativas, HTTP resulta el más pesado en tamaño de mensaje, consumo de energía, ancho de banda y latencia, CoAP el más liviano y MQTT ocupa una posición intermedia (esas comparaciones no consideran las retransmisiones) [15].
4. **Confiabilidad.** HTTP depende exclusivamente de la confiabilidad de TCP. MQTT le agrega tres niveles de calidad de servicio y el mensaje de última voluntad, mientras que CoAP ofrece, mediante sus mensajes no confirmables y confirmables, dos niveles aproximadamente equivalentes a QoS 0 y QoS 1 [15]. Ante pérdidas, MQTT presenta menor retardo que CoAP cuando las pérdidas son bajas y mayor cuando son elevadas, y con mensajes pequeños y pérdidas de hasta el 25% CoAP genera menos tráfico adicional para garantizar la entrega [16]. En un estudio sobre la comunicación entre vehículos y la nube, CoAP obtuvo las menores latencias y la menor sobrecarga, pero no logró entregar todos los mensajes con pérdidas elevadas, ni siquiera con mensajes confirmables, mientras que los protocolos basados en TCP, entre ellos MQTT, los entregaron todos [17].
5. **Paso por NAT y firewalls.** Muchos firewalls están configurados para bloquear el tráfico UDP [2]. Entre el 2% y el 4% de las redes de acceso terrestres lo bloquean por completo, y las restricciones se concentran en las redes empresariales; además, los NAT descartan las asociaciones UDP mucho antes que las TCP (en promedio, a los 160 segundos frente a los 386 minutos), lo que obliga a enviar mensajes de mantenimiento con mayor frecuencia [18]. Esto

BORRADOR

favorece a HTTP y a MQTT frente a CoAP, que para estos casos cuenta con una variante sobre TCP, TLS y WebSocket, a costa de un mayor tamaño de código, más memoria y más intercambios [18].

6. **Seguridad.** HTTP y MQTT se protegen con TLS sobre TCP, mientras que CoAP utiliza DTLS sobre UDP [15]. MQTT, además, deja a cada implementación la elección de los mecanismos de autenticación y autorización [5].
7. **Interoperabilidad y formato de los datos.** HTTP es el protocolo más interoperable, ya que basta con disponer de una pila HTTP en cada extremo [15], y CoAP se integra con la Web mediante *proxies* que traducen a HTTP [13]. MQTT, en cambio, requiere un broker y no indica en sus mensajes el formato del contenido, por lo que todos los clientes deben conocerlo de antemano [15]; la versión 5.0 agrega propiedades opcionales para declararlo, pero no impone ninguno [5]. Ninguno de los tres protocolos impone un mecanismo de serialización, aspecto que se analiza en la siguiente subsección.

5.1.4. Ventajas y desventajas

Frente a HTTP, la principal ventaja de MQTT es que el broker puede entregarle datos al dispositivo sin que este los solicite, lo que evita el *polling*. A esto se suman encabezados mínimos, una única conexión que se mantiene abierta y sirve en ambos sentidos, niveles de calidad de servicio propios y la posibilidad de conservar los mensajes de un cliente desconectado. Como contrapartida, MQTT es menos interoperable que HTTP, requiere un broker, que agrega un salto a cada mensaje y concentra la carga de todos los participantes, como se señaló al describir el modelo publicador/suscriptor, y obliga a que los clientes acuerden de antemano el formato de los datos.

Frente a CoAP, la ventaja de MQTT es la confiabilidad: al funcionar sobre TCP, en los estudios citados entregó todos los mensajes aun con pérdidas elevadas, y atraviesa con menos dificultad los NAT y firewalls, y su broker ofrece un desacoplamiento completo entre publicadores y suscriptores, que las notificaciones de mejor esfuerzo de Observe no alcanzan. Como contrapartida, CoAP es más liviano: genera menos sobrecarga, no requiere la pila TCP y admite solicitudes *multicast* [13], para las que MQTT no está diseñado [15]. Para redes en las que TCP no es viable, como algunas redes inalámbricas de sensores, existe una variante, MQTT-SN, que funciona sobre un servicio de datagramas y se conecta al broker mediante un *gateway* [19].

En síntesis, MQTT resulta preferible cuando se necesita una entrega confiable de datos en ambos sentidos sobre redes que no se controlan, con dispositivos detrás de NAT o firewalls; CoAP conviene para dispositivos muy restringidos que envían actualizaciones pequeñas sin necesidad de entrega garantizada [17]; y HTTP, para intercambios puntuales iniciados por el cliente o cuando la integración con la Web es prioritaria. La Tabla 2 resume la comparación.

BORRADOR

Tabla 2: Comparación entre HTTP, CoAP y MQTT.

Criterio	HTTP	CoAP	MQTT
Modelo de interacción	Solicitud y respuesta	Solicitud y respuesta (con Observe, notificaciones de mejor esfuerzo)	Publicador/suscriptor
Entrega sin solicitud previa	No (requiere *polling*)	Solo con Observe	Sí, a través del broker
Clientes desconectados	Sin entrega posterior	Sin entrega posterior	Sesiones persistentes y mensajes retenidos
Transporte	TCP	UDP	TCP
Encabezado	Texto, de longitud variable	Binario, 4 bytes fijos	Binario, 2 bytes fijos como mínimo
Confiabilidad	La de TCP	Mensajes confirmables y no confirmables	La de TCP y QoS 0, 1 y 2
Paso por NAT y firewalls	Favorable (TCP)	Desfavorable (UDP)	Favorable (TCP)
Seguridad	TLS	DTLS	TLS
Interoperabilidad	La más alta	Alta, mediante *proxies* a HTTP	Requiere un broker y un formato acordado

5.2. Protocol Buffers y JSON

Se definirá brevemente JSON, para posteriormente contrastarlo con Protocol Buffers.

5.2.1. JSON

JSON (*JavaScript Object Notation*) es un formato de intercambio de datos liviano, basado en texto e independiente del lenguaje de programación, derivado del estándar del lenguaje ECMAScript [20]. Representa los datos mediante objetos, que son conjuntos de pares nombre-valor, y arreglos, cuyos elementos pueden ser cadenas de texto, números, valores booleanos, el valor nulo u otros objetos y arreglos. Fuera de un ecosistema cerrado, los textos JSON deben codificarse en UTF-8 [20]. Por ejemplo, una lectura de un sensor con su temperatura y su nivel de batería puede representarse así:

```
{"temperatura":23.5,"bateria":80}
```

Como cada mensaje incluye los nombres de sus campos, JSON es autodescriptivo: puede leerse e interpretarse sin conocer de antemano la estructura de los datos, del mismo modo que los mensajes de HTTP/1.1, escritos en texto legible [2]. Esa característica determina su principal ventaja y su principal desventaja:

- **Ventaja:** cualquier persona o programa puede leer el mensaje sin un acuerdo previo sobre su estructura, lo que facilita la depuración y la interoperabilidad.
- **Desventaja:** los nombres de los campos viajan en cada mensaje y los números se representan como texto decimal. El ejemplo anterior ocupa 33 bytes, de los cuales 22 corresponden a los nombres de los campos y sus comillas. Además, la especificación recomienda no esperar más precisión que la de un número de punto flotante de doble precisión, por lo que solo garantiza la interoperabilidad de los enteros de hasta $2^{53} - 1$ en valor absoluto [20].

BORRADOR

5.2.2. Contraste con Protocol Buffers

De la descripción anterior, de la presentada para Protocol Buffers y de los estudios que comparan ambos mecanismos se desprenden las siguientes diferencias:

1. **Representación.** JSON codifica los datos como texto, mientras que Protocol Buffers utiliza un formato binario en el que cada campo se codifica como un registro de tipo, longitud y valor, identificado por un número de campo en lugar de su nombre [9].
2. **Esquema compartido.** JSON no requiere un esquema previo. Protocol Buffers, en cambio, requiere que emisor y receptor compartan el archivo `.proto` que define la estructura de los mensajes, ya que sin él no es posible interpretar los datos recibidos. A cambio, ese esquema permite generar código para cada lenguaje y fijar el tipo de cada campo [6].
3. **Tamaño de los mensajes.** Al reemplazar los nombres por números de campo y codificar los valores en binario, Protocol Buffers produce mensajes considerablemente más pequeños. En un relevamiento sobre 27 documentos JSON de distintos ámbitos, Protocol Buffers redujo el tamaño con una mediana del 70,6% [21]; con cargas reales tomadas de mensajes MQTT públicos, el tamaño medio pasó de 111 bytes en JSON a 40 bytes [22]; y con mensajes de producción de vehículos conectados, de 12.187 a 1.157 bytes en promedio [17]. La reducción, sin embargo, no es universal: en uno de los 27 documentos, formado por arreglos muy anidados y casi sin nombres de campo, Protocol Buffers resultó más grande que JSON [21].
4. **Costo de procesamiento.** Aunque la documentación presenta a Protocol Buffers como más rápido que JSON [6], en microcontroladores la diferencia no es clara: en algunas arquitecturas, la biblioteca de JSON evaluada serializó más rápido. Aun así, con radios lentas transmitir un byte consume unas diez mil veces más energía que un ciclo de CPU, por lo que Protocol Buffers resultó la opción más eficiente en energía, a costa de un mayor uso de memoria RAM [22].
5. **Tipado y evolución.** JSON ofrece solo tipos genéricos y no impone una estructura, por lo que nada garantiza que emisor y receptor interpreten los datos de la misma manera. Protocol Buffers asigna un tipo concreto a cada campo y controla la evolución de los mensajes: el código nuevo puede leer mensajes antiguos, y los campos ausentes toman un valor por defecto [6].
6. **Estandarización y alcance.** JSON es un estándar del IETF [20], mientras que Protocol Buffers no es un estándar formal de ningún organismo, lo que puede ser una limitación en entornos que exigen basarse en estándares. Además, Protocol Buffers supone que cada mensaje entra completo en memoria, por lo que no resulta adecuado para datos de más de unos pocos megabytes [6].

5.2.3. Ventajas y desventajas

La principal ventaja de Protocol Buffers frente a JSON es el tamaño de los mensajes, que en los estudios citados resulta de casi tres a más de diez veces menor. Esta diferencia es especialmente relevante en los sistemas considerados, que envían mensajes pequeños a alta frecuencia por enlaces limitados: de poco sirve que MQTT o CoAP reduzcan sus encabezados a unos pocos bytes si la carga útil se codifica en texto. A esto se suman el tipado de los campos y la evolución controlada del esquema. Como contrapartida, Protocol Buffers sacrifica la legibilidad, obliga a distribuir y mantener el esquema entre todos los participantes, lo que introduce un acoplamiento en el formato de los datos que el modelo publicador/suscriptor no elimina, y no es un estándar formal. JSON, por su parte, conviene cuando la legibilidad, la depuración o la integración con sistemas existentes, como las aplicaciones web, pesan más que el tamaño de los mensajes.

BORRADOR

En síntesis, Protocol Buffers resulta preferible cuando el tamaño de los mensajes y el costo de transmitirlos son críticos y es posible distribuir un esquema común entre los participantes, mientras que JSON conviene cuando se priorizan la legibilidad, la flexibilidad y la interoperabilidad sin acuerdos previos. La Tabla 3 resume la comparación.

Tabla 3: Comparación entre JSON y Protocol Buffers.

Criterio	**JSON**	**Protocol Buffers**
Representación	Texto (UTF-8)	Binaria
Esquema	No requiere	Requiere un archivo `.proto` compartido
Autodescriptivo	Sí: incluye los nombres de los campos	No: incluye números de campo
Legibilidad	Legible por personas	Requiere herramientas y el esquema
Números	Texto decimal	Binarios, de longitud variable o fija
Tamaño de los mensajes	Mayor	De casi tres a más de diez veces menor
Tipado	Genérico	Un tipo concreto por campo
Evolución	Sin control	Controlada por números de campo y valores por defecto
Estandarización	Estándar del IETF	Sin estandarización formal

BORRADOR

Desarrollo

1. MQTT sobre TCP y TLS

1.1. Conexión persistente y keep-alive

1.2. Cifrado con TLS

1.3. QoS MQTT sobre la confiabilidad de TCP

2. Capa de red

2.1. NAT y firewalls

2.2. Ruteo interno

2.3. Ruteo externo

2.4. Convergencia y conexiones de larga duración

3. Ruteo por tópicos y ruteo IP

4. Casos de uso

5. Diseño experimental

5.1. Plataforma común

Los dos experimentos corren sobre el mismo entorno, montado en una sola computadora, donde cada nodo es un ESP32 emulado con QEMU.

El ESP32 emulado no tiene WiFi. En su lugar, QEMU le provee una placa Ethernet emulada, que el firmware maneja como cualquier otra interfaz. Cada instancia usa la red de usuario de QEMU, que no requiere permisos de administrador: desde el nodo, la computadora anfitriona se ve en la dirección `10.0.2.2`. Ahí corre el broker Mosquitto, al que se conectan todos los nodos. Para varios nodos se lanzan varias instancias de QEMU en paralelo, cada una con su propia copia de la imagen de flash.

La emulación impone algunas restricciones. Los tiempos de ejecución no son los de una placa real, porque dependen de la carga de la computadora anfitriona. Además, la red de usuario no crea una interfaz en la computadora anfitriona, así que no se puede degradar el enlace de un nodo por separado. Para introducir latencia o pérdida en cada nodo hace falta conectar cada instancia a una interfaz virtual propia unida a un puente, y aplicar `netem` sobre esa interfaz.

BORRADOR

5.2. Experimento 1: MQTT contra HTTPS para telemetría

5.2.1. Planteo

5.2.2. Implementación

5.2.3. Resultados

5.2.4. Análisis de resultados

5.3. Experimento 2: sincronización de osciladores acoplados

5.3.1. Planteo

El modelo. Cada oscilador tiene una frecuencia natural ω_i, que es fija, y una fase θ_i, que evoluciona según

$$\dot{\theta}_i = \omega_i + \frac{K}{N}\sum_{j=1}^{N}\sin\big(\theta_j - \theta_i\big), \quad i = 1, ..., N \tag{1}$$

donde K es la intensidad del acoplamiento, en rad/s. El acoplamiento cambia la velocidad instantánea $\dot{\theta}_i$: si el oscilador j va adelantado, $\sin\big(\theta_j - \theta_i\big) > 0$ y el oscilador i se acelera; si va atrasado, se frena.

Parámetro de orden y campo medio. El grado de sincronía se mide con el centroide de las fases sobre el círculo unitario:

$$re^{i\psi} = \frac{1}{N}\sum_{j=1}^{N} e^{i\theta_j} \tag{2}$$

r vale 0 si las fases están dispersas y 1 si coinciden; ψ es la fase media. Con esta definición, la Ecuación 1 se reescribe en forma de campo medio, que vale para cualquier N:

$$\dot{\theta}_i = \omega_i + Kr\sin(\psi - \theta_i) \tag{3}$$

Cada oscilador es atraído hacia la fase media con una fuerza Kr. Es la forma que se implementa en los nodos. El promedio de fases tiene que ser circular: se suman los vectores $\big(\cos\theta_j, \sin\theta_j\big)$ y se toman el módulo y el ángulo de la suma. Promediar los ángulos directamente falla en el corte entre 2π y 0.

Frecuencia común. Como el acoplamiento es simétrico, al sumar todas las ecuaciones los senos se cancelan de a pares:

$$\frac{d}{dt}\sum_i \theta_i = \sum_i \omega_i \quad \Rightarrow \quad \Omega = \overline{\omega} = \frac{1}{N}\sum_i \omega_i \tag{4}$$

Si el grupo se engancha, gira a la frecuencia media Ω. Desde ahí se trabaja con las desintonías $\Delta_i = \omega_i - \overline{\omega}$. Con retardo en el acoplamiento, esta propiedad deja de valer.

Límite de infinitos osciladores. Con frecuencias tomadas de una distribución $g(\omega)$ unimodal y simétrica, aparece una transición en el acoplamiento crítico

$$K_c = \frac{2}{\pi g(0)} \tag{5}$$

BORRADOR

Para una distribución de Lorentz de ancho γ, $K_c = 2\gamma$ y, por encima del umbral, $r = \sqrt{1 - K_c/K}$. Con pocos nodos este resultado queda solo como contexto, y K_c se calcula como se muestra a continuación.

Umbral exacto con N finito. En el marco que gira a $\overline{\omega}$, un estado enganchado cumple

$$\Delta_i = u \sin(\theta_i - \psi), \quad u = Kr \tag{6}$$

Cada oscilador necesita $|\Delta_i| \leq u$ y queda con un desfasaje $\theta_i - \psi = \arcsin(\Delta_i/u)$. Reemplazando en la Ecuación 2, con todos los cosenos positivos, se obtiene la condición de autoconsistencia:

$$r = \frac{u}{K} = f(u) = \frac{1}{N} \sum_{i=1}^{N} \sqrt{1 - \left(\frac{\Delta_i}{u}\right)^2}, \quad u \geq \max_i |\Delta_i| \tag{7}$$

El estado enganchado existe si K alcanza el mínimo de $u/f(u)$:

$$K_c = \min_{u \geq \max_i |\Delta_i|} \frac{u}{f(u)} \tag{8}$$

Con esto se calculan, para las frecuencias concretas de cada corrida, el umbral de enganche total, el r final y el desfasaje de cada nodo. Con $N = 2$ se obtiene $K_c = |\Delta\omega|$.

$N = 3$ simétrico. Con frecuencias $\overline{\omega} - \Delta$, $\overline{\omega}$ y $\overline{\omega} + \Delta$, por simetría el oscilador central queda alineado con ψ y los otros dos a $\pm\alpha$. La condición de equilibrio del oscilador rápido es

$$\Delta = \frac{K}{3}(\sin\alpha + \sin 2\alpha) \tag{9}$$

El máximo de $\sin\alpha + \sin 2\alpha$ es aproximadamente 1,760, en $\alpha \approx 53{,}6°$, de modo que $K_c \approx 1{,}704\ \Delta$. El oscilador central hace de puente entre los extremos.

Enganche parcial y normalización. Con $N \geq 3$ y frecuencias asimétricas, puede engancharse un subgrupo mientras otro nodo deriva. Este caso no tiene fórmula simple y se estudia por simulación, comparando las frecuencias efectivas de cada nodo. Como el acoplamiento se divide por N, al agregar nodos con el mismo K cada interacción individual se debilita. Si cada nodo dividiera por la cantidad de fases recibidas en lugar de un N fijo, el modelo cambiaría cuando se pierden mensajes; por eso conviene usar N fijo.

Efecto de la red: retardo. En la red, cada nodo conoce las fases de los demás con un retardo τ:

$$\dot{\theta}_i = \omega_i + \frac{K}{N} \sum_{j \neq i} \sin\big(\theta_j(t - \tau) - \theta_i(t)\big) \tag{10}$$

La suma excluye $j = i$ porque cada nodo conoce su propia fase sin demora. Para osciladores idénticos, si se incluyera ese término propio, la frecuencia común cumpliría $\Omega = \omega - K\sin(\Omega\tau)$, y el estado en fase sería estable si $K\cos(\Omega\tau) > 0$. En la implementación ese término no existe.

$N = 2$ con retardo. Con $\varphi = \theta_2 - \theta_1$ y $\Delta\omega = \omega_2 - \omega_1$, el estado enganchado de la Ecuación 10 cumple

$$\Delta\omega = K \sin\varphi \cos(\Omega\tau), \quad \Omega = \overline{\omega} - \frac{K}{2} \cos\varphi \sin(\Omega\tau) \tag{11}$$

La primera condición solo tiene solución si $K\ |\cos(\Omega\tau)| \geq |\Delta\omega|$. Aproximando $\Omega \approx \overline{\omega}$:

BORRADOR

$$K_c(\tau) \approx \frac{|\Delta\omega|}{|\cos(\overline{\omega}\tau)|} \tag{12}$$

Una simulación con $\overline{\omega} = 2\pi$ rad/s y $\Delta\omega = 0{,}2$ rad/s dio el primer K con enganche en 0,200 sin retardo, 0,247 con $\tau = 100$ ms y 0,647 con $\tau = 200$ ms, y la frecuencia común bajó de 6,283 a 6,178 rad/s. La Ecuación 12 da así una predicción de K_c y de Ω en función de la latencia.

El retardo entra multiplicado por la frecuencia absoluta de parpadeo $\overline{\omega}$, no por la desintonía. A 1 Hz, con 250 ms de latencia ya no hay enganche con un K moderado; a 0,5 Hz el límite pasa a 500 ms. La frecuencia de parpadeo define cuánta latencia tolera el sistema.

Pérdida de paquetes. MQTT viaja sobre TCP: si se descarta un paquete, TCP lo retransmite y el mensaje llega igual, más tarde, aun con QoS 0. La pérdida de paquetes se traduce en retardo adicional y jitter, no en mensajes perdidos. Los mensajes se pierden de verdad solo si se descartan en la aplicación o si se corta la conexión. En ese caso, el efecto depende de la política del nodo:

- si mantiene la última fase recibida, la pérdida aumenta la edad de la información y actúa como retardo adicional;
- si usa solo lo recibido en cada paso, con N fijo, el acoplamiento efectivo baja a aproximadamente $K(1-p)$.

Actualización discreta. Cada nodo integra su fase con un paso propio y publica con un período h. Son dos parámetros distintos. Con integración de Euler, el lazo es estable si $0 < Kh < 2$, pero con valores reales $Kh \approx 0{,}05$, así que ese límite no influye. El período de publicación determina cada cuánto llega información nueva, y actúa como un retardo adicional de alrededor de $h/2$.

Jitter y caída del broker. La variación del retardo actúa como ruido en la fase: baja r en régimen y sube el K_c medido. Si cae el broker, el acoplamiento se anula y cada nodo vuelve a su frecuencia natural.

Acoplamiento efectivo. K es una ganancia del software de cada nodo: se fija al comienzo de cada corrida y no cambia durante ella. No depende de la red y no cambia el tráfico, que lo determinan h y el tamaño de los mensajes. La red no modifica K, pero sí el acoplamiento efectivo (Tabla 4). Por eso hace falta un K mayor que el teórico para lograr el enganche, y la diferencia entre el K_c teórico y el medido es lo que el stack le quita a la sincronización.

Tabla 4: Efecto de la red sobre el acoplamiento efectivo.

Efecto de la red	**Acoplamiento efectivo**
Latencia τ	Con $N = 2$, el enganche exige $K\,\lvert\cos(\overline{\omega}\tau)\rvert \geq \lvert\Delta\omega\rvert$
Pérdida de mensajes (solo lo recibido, N fijo)	Aproximadamente $K(1-p)$
Período de publicación h	Información vieja: retardo adicional de alrededor de $h/2$
Caída del broker	Nulo

5.3.2. Implementación

5.3.3. Resultados

5.3.4. Análisis de resultados

BORRADOR

Conclusiones

BORRADOR

Bibliografía

[1] D. E. Comer, *Internetworking with TCP/IP, Volume One: Principles, Protocols, and Architecture*, 6.ª ed. Upper Saddle River, NJ: Pearson, 2014.

[2] J. F. Kurose y K. W. Ross, *Redes de computadoras. Un enfoque descendente*, 7.ª ed. Madrid: Pearson Educación, 2017.

[3] P. T. Eugster, P. A. Felber, R. Guerraoui, y A.-M. Kermarrec, «The many faces of publish/subscribe», *ACM Computing Surveys*, vol. 35, n.º 2, pp. 114-131, 2003, doi: 10.1145/857076.857078.

[4] W. Stallings, *Comunicaciones y redes de computadores*, 7.ª ed. Madrid: Pearson Educación, 2004.

[5] A. Banks, E. Briggs, K. Borgendale, y R. Gupta, Eds., «MQTT version 5.0», OASIS, OASIS Standard, 2019. [En línea]. Disponible en: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[6] Google, «Overview». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/overview/

[7] protocolbuffers, «protobuf: Protocol Buffers - Google's data interchange format». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://github.com/protocolbuffers/protobuf

[8] MQTTX Team, «How to Publish and Receive Protobuf Messages within MQTT?». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://www.emqx.com/en/blog/how-to-publish-and-receive-protobuf-messages-within-mqtt

[9] Google, «Encoding». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/programming-guides/encoding/

[10] Google, «Language Guide (proto 3)». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/programming-guides/proto3/

[11] R. T. Fielding, M. Nottingham, y J. Reschke, Eds., «HTTP Semantics», RFC 9110, 2022. doi: 10.17487/RFC9110.

[12] S. Loreto, P. Saint-Andre, S. Salsano, y G. Wilkins, «Known Issues and Best Practices for the Use of Long Polling and Streaming in Bidirectional HTTP», RFC 6202, 2011. doi: 10.17487/RFC6202.

[13] Z. Shelby, K. Hartke, y C. Bormann, «The Constrained Application Protocol (CoAP)», RFC 7252, 2014. doi: 10.17487/RFC7252.

[14] K. Hartke, «Observing Resources in the Constrained Application Protocol (CoAP)», RFC 7641, 2015. doi: 10.17487/RFC7641.

[15] N. Naik, «Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP», en *2017 IEEE International Systems Engineering Symposium (ISSE)*, Vienna, Austria: IEEE, 2017. doi: 10.1109/SysEng.2017.8088251.

[16] D. Thangavel, X. Ma, A. Valera, H.-X. Tan, y C. K.-Y. Tan, «Performance evaluation of MQTT and CoAP via a common middleware», en *2014 IEEE Ninth International*

BORRADOR

Conference on Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP), Singapore: IEEE, 2014. doi: 10.1109/ISSNIP.2014.6827678.

[17] D. P. Proos y N. Carlsson, «Performance comparison of messaging protocols and serialization formats for digital twins in IoV», en *2020 IFIP Networking Conference (Networking)*, Paris, France: IFIP, 2020.

[18] C. Bormann, S. Lemay, H. Tschofenig, K. Hartke, B. Silverajan, y B. Raymor, «CoAP (Constrained Application Protocol) over TCP, TLS, and WebSockets», RFC 8323, 2018. doi: 10.17487/RFC8323.

[19] A. Stanford-Clark y H. L. Truong, «MQTT For Sensor Networks (MQTT-SN) Protocol Specification, Version 1.2», Especificación técnica de IBM, 2013.

[20] T. Bray, Ed., «The JavaScript Object Notation (JSON) Data Interchange Format», RFC 8259, 2017. doi: 10.17487/RFC8259.

[21] J. C. Viotti y M. Kinderkhedia, «A Benchmark of JSON-compatible Binary Serialization Specifications», *arXiv preprint arXiv:2201.03051*, 2022, doi: 10.48550/arXiv.2201.03051.

[22] D. Friesel y O. Spinczyk, «Data Serialization Formats for the Internet of Things», *Electronic Communications of the EASST*, vol. 80, 2021, doi: 10.14279/tuj.eceasst.80.1134.

[23] L. Zhang, «Building Facebook Messenger». [En línea]. Disponible en: https://engineering.fb.com/2011/08/12/android/building-facebook-messenger/

[24] InOrbit, «InOrbit.AI: We make robots work». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://www.inorbit.ai/

[25] inorbit-ai, «edge-sdk-python: InOrbit Python Edge SDK». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://github.com/inorbit-ai/edge-sdk-python

[26] OpenRobOps, «OpenRobOps: Open source robot fleet management». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://openrobops.org/

[27] OpenRobOps/oro, «oro: OpenRobOps open-source robot fleet management and operations platform». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://github.com/OpenRobOps/oro

BORRADOR

Anexos

A. Mensajes del experimento de sincronización

BORRADOR